'Hello World' will be returned by the server to the user, because in Ruby, the last statement is treated as the return statement.

To run the code, type the following command:

```
ruby hello.rb
```

The following output should be visible with some variation, depending on your version of Ruby and the backend server:

```
== Sinatra (v1.4.8) has taken the stage on 4567 for
development with backup from Puma

Puma starting in single mode...
* Version 3.6.2 (ruby 2.2.4-p230), codename: Sleepy Sunday
Serenity
* Min threads: 0, max threads: 16
* Environment: development
* Listening on tcp://localhost:4567
Use Ctrl-C to stop
```

If you open *http://localhost:4567* you will see a 'Hello World' message.

Sinatra has a *params* hash to capture parameters passed to the route. A small excerpt is shown below:

```
post '/secret' do
"Message is  #{params[:message]} "
end
```

The following code displays the parameter named *message* which was POSTed to */secret*. All the parameters sent to a route are available in hash named *params*. If we make a POST request with the message 'meet me at Coffee Shop' as follows:

```
curl --request POST   --url 'http://localhost:4567/
secret?message=meet%20me%20at%20Coffee%20shop'
```

…we will get the following output:

```
Message is meet me at Coffee shop
```

Sinatra routes can contain regular expressions, named parameters and wild card expressions, as well.

```
get '/send/:room/?' do
 "Sending message to room no #{params[:room]}"
end
```

This time we don't need to pass the parameter explicitly; it has become a part of the URL itself and it will just work, and like in the previous instance, the parameters are available via *params* hash. The interesting bit in the route is '/?'; it is used so that the route will work with and without the trailing slash, so that both *http://localhost:4567/42* and *http://localhost:456/42/* are legal.

```
curl --request GET --url http://localhost:4567/send/42/

Sending message to room no 42
```

Splat (*), or what is commonly known as the wildcard operator, will allow anything and is accessible via a special parameter named *splat* which is an array of all the values captured by splat against *.

```
get '/mix/*/with/*/?' do
"I have #{params[:splat]} and I am going to mix
#{params[:splat].first} with #{params[:splat].last} "

end

curl --request GET --url http://localhost:4567/mix/apple/
with/oranges/

I have ["apple", "oranges"] and I am going to mix apple
with oranges
```

There are many ways to play with routes, but covering them all would be beyond the scope of this article.

One cool thing that you can quickly do with Sinatra is make APIs, which generally don't require any interface and most of them are stateless. In the following example, we will make a simple API that returns all environment variables available on the machine in JSON format. For JSON conversion, we need a JSON parser and generator, which Ruby already ships with.

Content type tells Sinatra to explicitly specify itself in the response header.

```
# Load the JSON library
require 'json'


get '/json' do
content_type :json
  ENV.to_h.to_json # To json converts the ENV which was
converted into hash to json
end

curl --request GET --url http://localhost:4567/json
```

The output may vary depending on the system and configuration:

```
{
  "PWD": "/home/jatin/Area_51/sinatra_example",
```